

Base de Datos Vectoriales

SEMANA 11

Heider Sanchez
hsanchez@utec.edu.pe

4.



Modelos de Recuperación Multimedia

Búsqueda Eficiente

Algoritmos de Búsqueda: exploración secuencial

- Búsqueda por rango

Algorithm RangeSearch(Q, r)

```
1.  result = [ ]
2.  for all objects  $C_i$  in the collection
3.      dist = Dist(Q,  $C_i$ )
4.      if dist < r
5.          append(result,  $C_i$ )
6.      endif
7.  endfor
8.  return result
```

$$O(N \times D^p)$$

N : # de elementos en la colección

D : dimensión del espacio
característico

p : indica el tipo de distancia (lineal,
cuadrática)

Algoritmos de Búsqueda: exploración secuencial

- Búsqueda de los k vecinos más cercanos

Algorithm KnnSearch(Q, k)

```
1. result = [ ]
2. for all objects  $C_i$  in the collection
3.     |   dist = Dist(Q,  $C_i$ )
4.     |   append(result, { $C_i$ , dist})
5. endfor
6. orderByDist(result)
7. return result[1:k]
```

$$O(N \times D^p) + O(N \log N)$$

N : # de elementos en la colección

D : dimensión del espacio característico

p : indica el tipo de distancia (lineal, cuadrática)

Algoritmos de Búsqueda: exploración secuencial

- La complejidad computacional esta en función de:
 - $O(N)$: La cantidad de elementos en la colección puede ser muy grande.
 - $O(D^p)$: La función de distancia puede ser cuadrática $p \geq 1$.
- Se requiere métodos eficientes para encontrar los objetos similares a una consulta.
 - Índices y algoritmos de búsqueda

Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar
 - La verdadera distancia entre objetos puede ser:
 - Una distancia métrica.
 - Una distancia no métrica.
 - Distancia verdadera (en general) costosa de calcular.
 - Para responder una consulta se utiliza un arquitectura multi-step
 - Objetos recuperados desde el índice no son el resultado, sino sólo candidatos.

Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar
 - El conjunto de candidatos debe ser pequeño con respecto al tamaño de la BD.
 - Selectividad del filtro

$$\sigma_F = \frac{\# \text{ de candidatos}}{\# \text{ de objetos en la BD}}$$

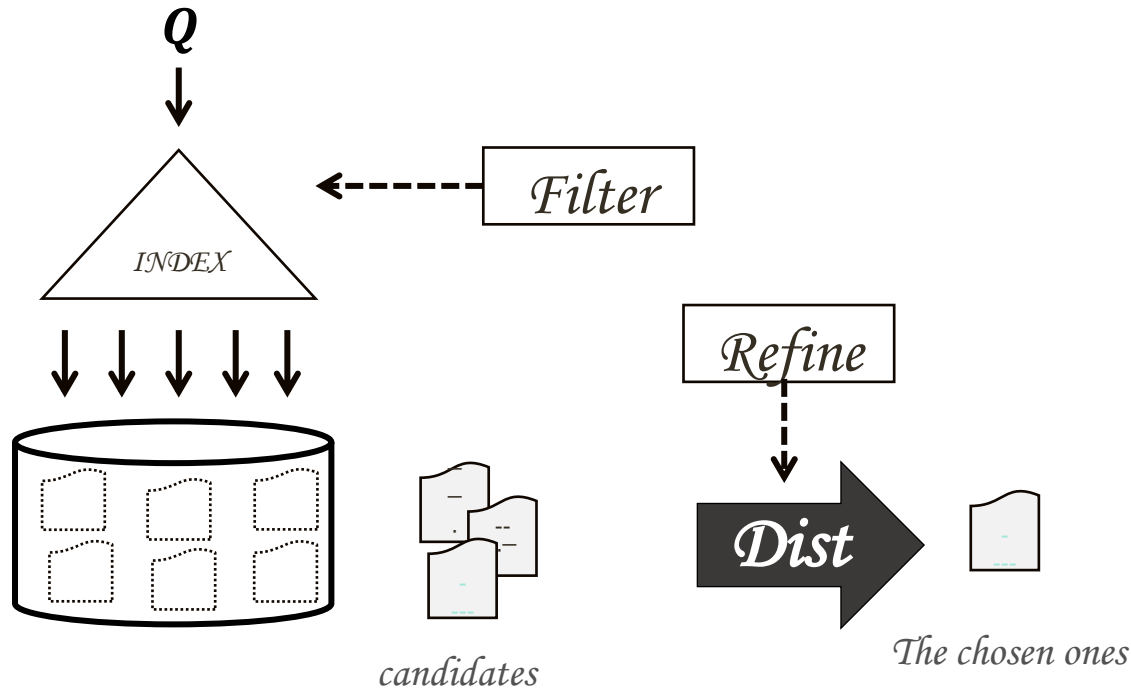
- Para los candidatos se mide la distancia verdadera con el objeto de consulta (es más cara, pero más selectiva).

Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar
 - Filtrado en cascada
 - Primer filtro determina candidatos con el índice
 - Filtros siguientes reducen el conjunto de candidatos
 - Paso de refinamiento determina rectitud de la respuesta

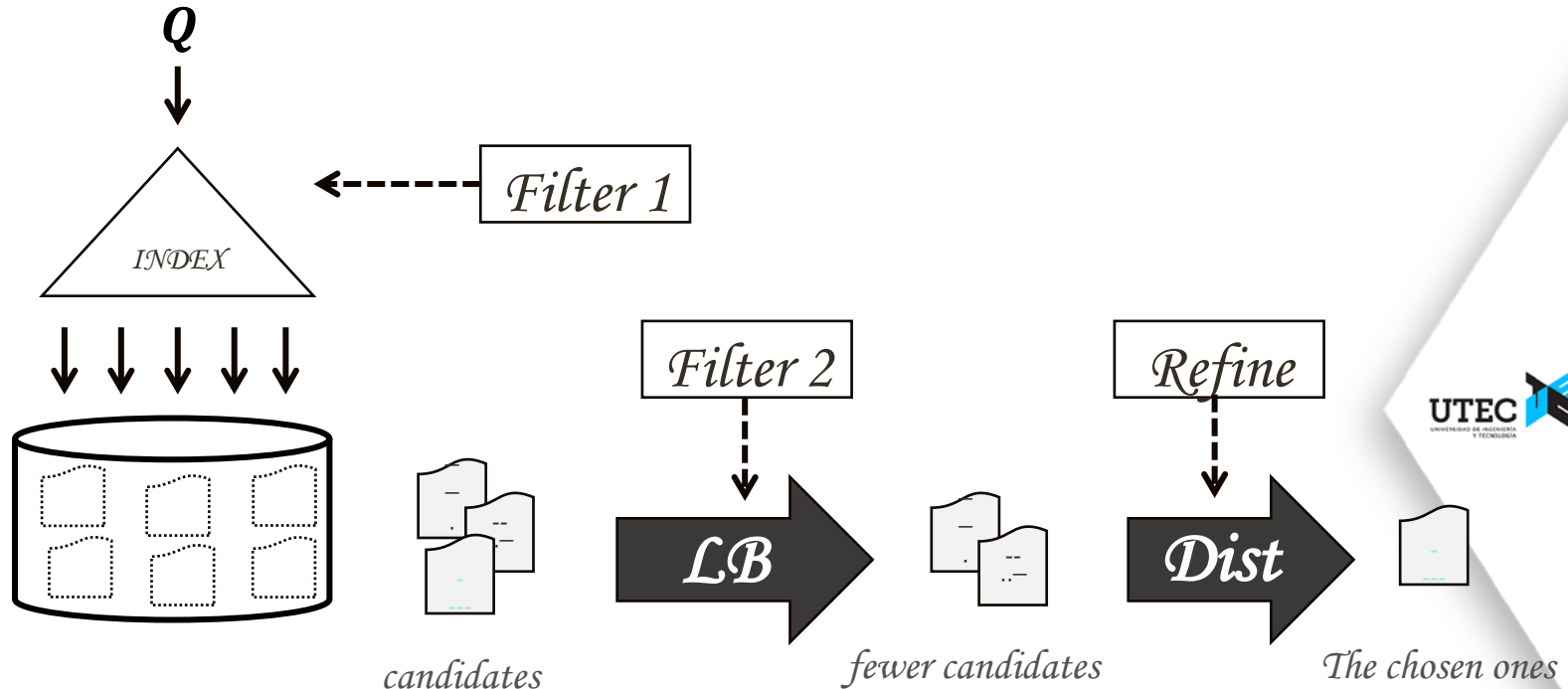
Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar



Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar

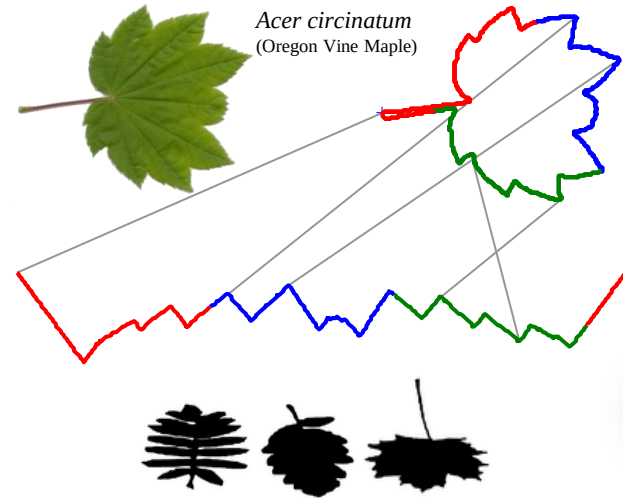
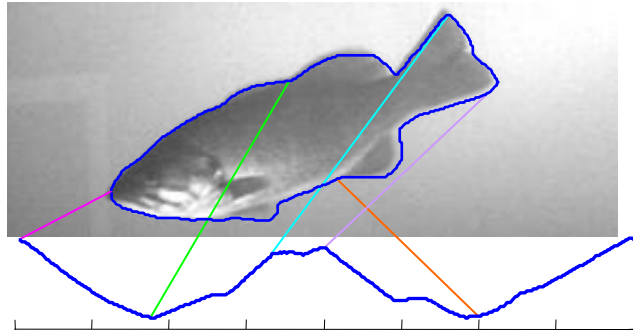


Algoritmos de Búsqueda: filtrar-y-refinar

- Método basado en filtrar-y-refinar
 - Caso ideal para filtrar-y-refinar
 - **Distancia filtro debe ser cota inferior (lower bound) de la distancia verdadera**
 - Así, se garantiza que no se pierda ningún objeto relevante (no hay falsos negativos)
 - Si el filtro no es cota inferior
 - Pueden haber falsos negativos
 - No se garantiza que el resultado esté completo

Algoritmos de Búsqueda: Lower Bounding Distance

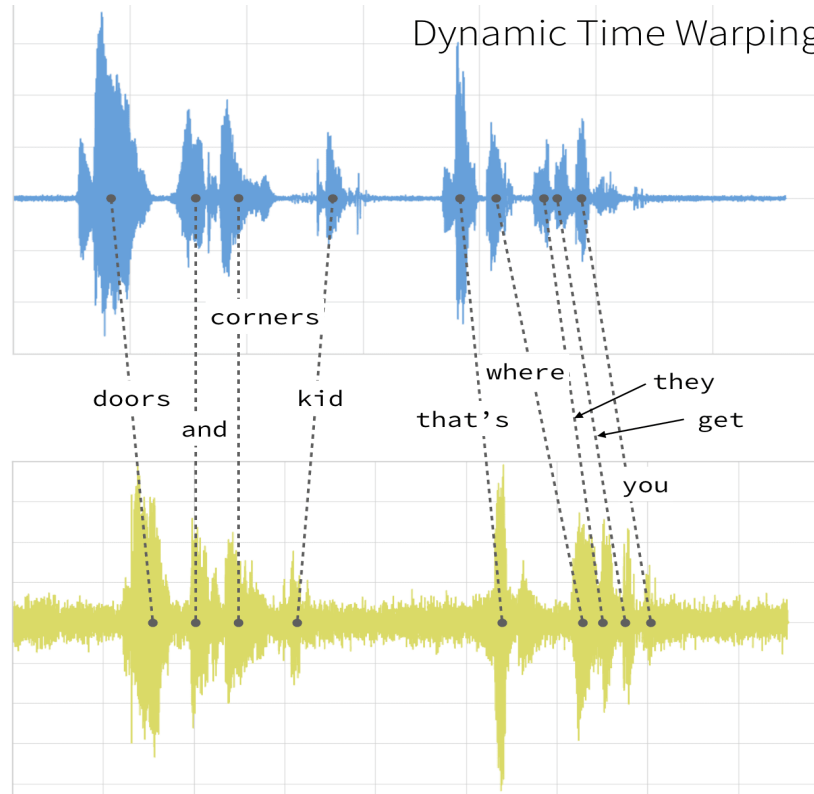
- Dynamic Time Warping distance



Algoritmos de Búsqueda: Lower Bounding Distance

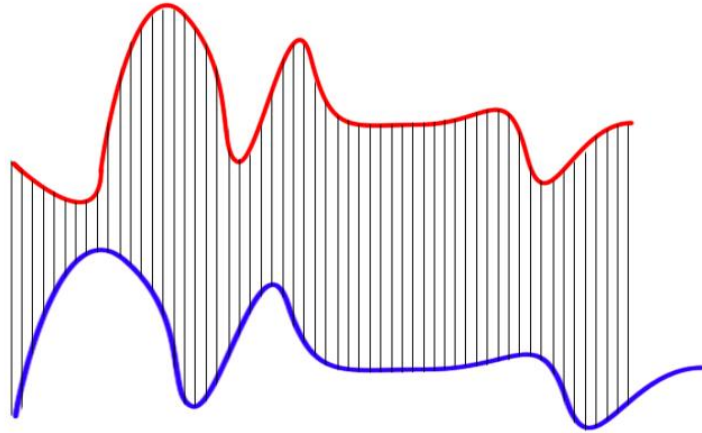
- Dynamic Time Warping distance

Speech matching

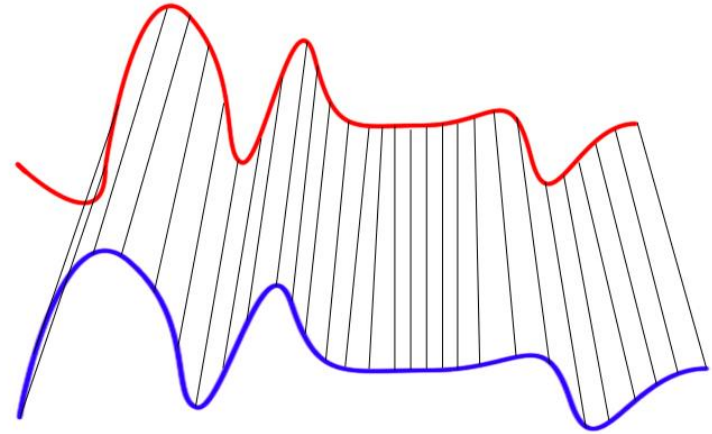


Algoritmos de Búsqueda: Lower Bounding Distance

- Dynamic Time Warping distance



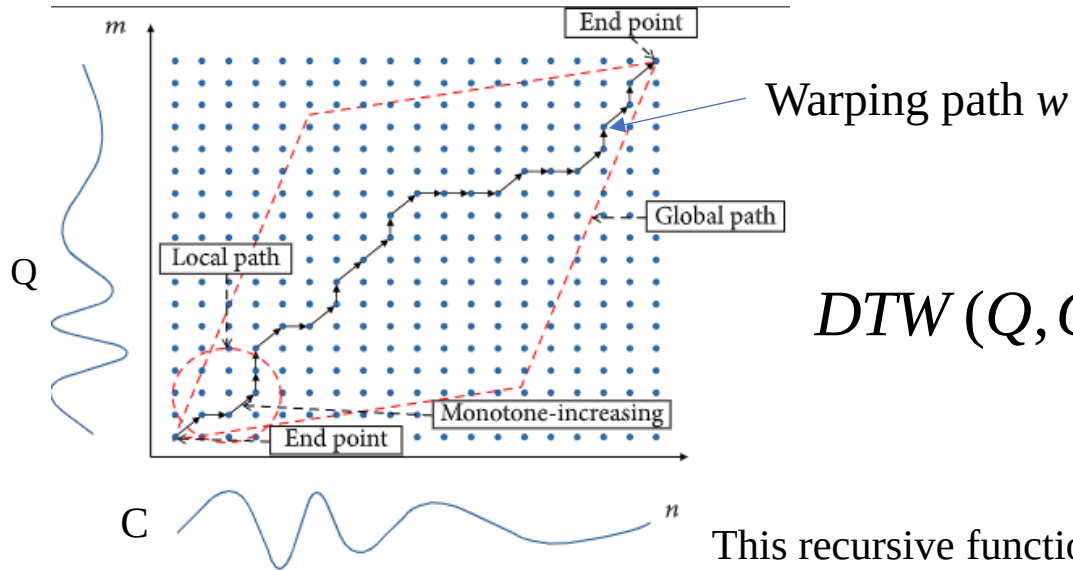
Euclidean Matching



Dynamic Time Warping Matching

Algoritmos de Búsqueda: Lower Bounding Distance

- Dynamic Time Warping distance



$$DTW(Q, C) = \min \left\{ \sqrt{\sum_{k=1}^K w_k} / K \right\}$$

This recursive function gives us the minimum cost path

$$M(i, j) = d(q_i, c_j) + \min \{ M(i-1, j-1), M(i-1, j), M(i, j-1) \}$$

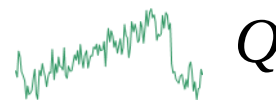
Algoritmos de Búsqueda: Lower Bounding Distance

- Dynamic Time Warping distance

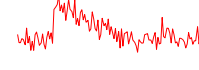
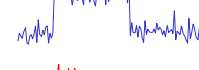
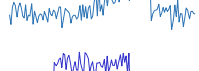
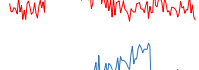
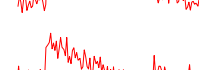
$$O(N \times D^2)$$

Algorithm Sequential_Scan(Q)

```
1. best_so_far = infinity;
2. for all sequences in database
3.   true_dist = DTW(Ci, Q);
4.   if true_dist < best_so_far
5.     best_so_far = true_dist;
6.     index_of_best_match = i;
7.   endif
8. endfor
```



Q



	DTW(Q, C _i)
	14.1
	1.7
	12.2
	17.9
	9.5
	14.2
	11.2

Algoritmos de Búsqueda: Lower Bounding Distance

- Lower bounding for the Dynamic Time Warping [KR05]

The lower bound function is very fast...

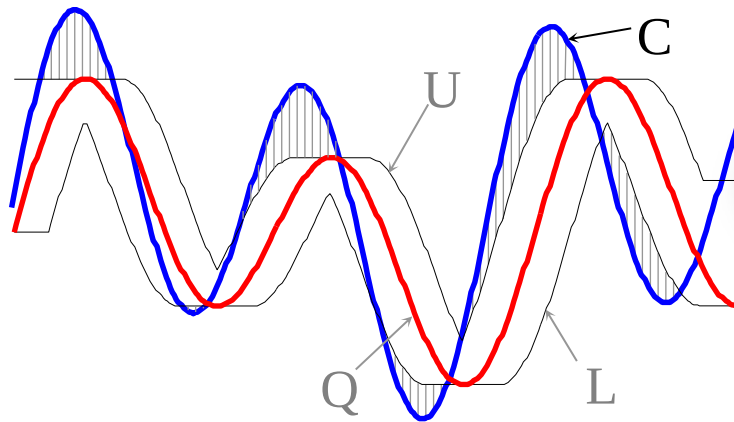
$$\text{LB_Keogh}(Q, C) \leq \text{DTW}(Q, C)$$

The true DTW function is very slow...

LB_Keogh

Envelope-Based Lower Bound

$$\text{LB_Keogh}(Q, C) = \sum_{i=1}^n \begin{cases} (c_i - U_i)^2 & \text{if } c_i > U_i \\ (c_i - L_i)^2 & \text{if } c_i < L_i \\ 0 & \text{otherwise} \end{cases}$$

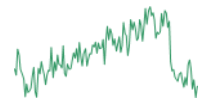


Algoritmos de Búsqueda: Lower Bounding Distance

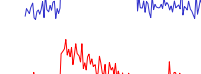
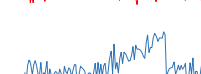
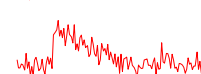
- Lower bounding for the Dynamic Time Warping

Algorithm Lower_Bounding_Sequential_Scan(Q)

```
1.  best_so_far = infinity;
2.  for all sequences in database
3.    LB_dist = LB_Keogh( $C_i$ , Q);
4.    if LB_dist < best_so_far
5.      true_dist = DTW( $C_i$ , Q);
6.      if true_dist < best_so_far
7.        best_so_far = true_dist;
8.        index_of_best_match = i;
9.      endif
10.   endif
11. endfor
```

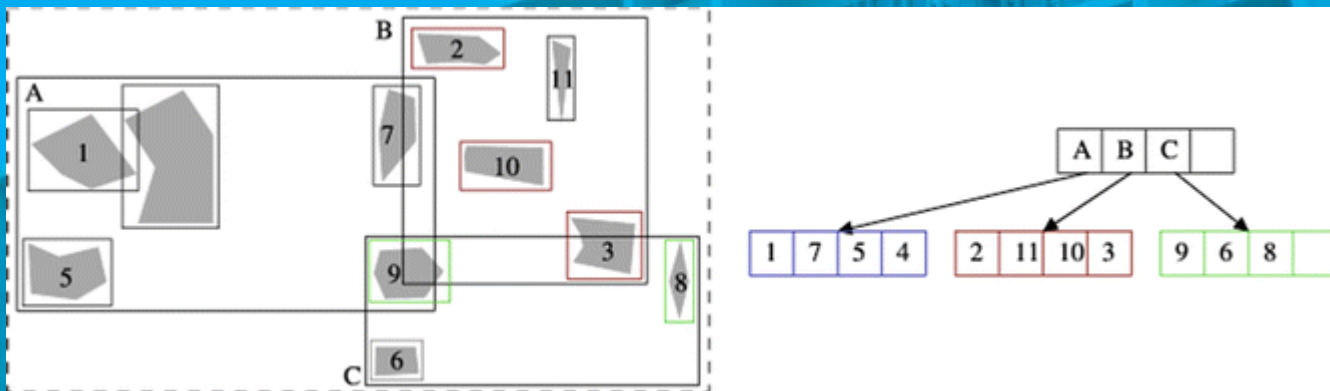


Q



LB	DTW(Q, C_i)
9.2	14.1
1.3	1.7
5.2	--
2.5	--
1.6	9.5
3.5	--
5.8	--

Índices Multidimensionales



Índices Multidimensionales

- **Región espacial**

- Garantizan que puntos cercanos se almacenen en lo posible en la misma página de datos o subárbol .
- Regiones de páginas jerárquicamente organizadas siempre deben estar contenidas completamente en la región de su padre.

- **Índices multidimensionales son dinámicos**

- Operaciones de inserción y borrado eficientes $O(\log n)$

Índices Multidimensionales con soporte KNN

- **R*-Tree:**

- Variante optimizada del R-Tree.
- Mejora el rendimiento mediante técnicas como la minimización del solapamiento de nodos.
- Soporta vectores característicos de **baja dimensión**.

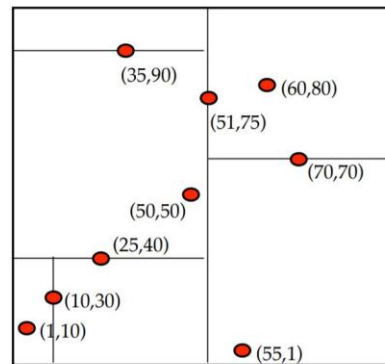
- **KD-Tree**

- Árbol binario para particionar el espacio en k dimensiones.
- Bueno para datos de baja a moderada dimensionalidad.

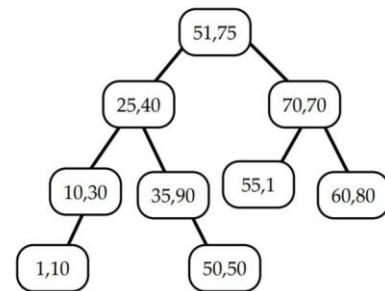
- **Ball Tree**

- Divide el espacio en hiperesferas (ball nodes).
- Mejor que KD-Tree en alta dimensionalidad.

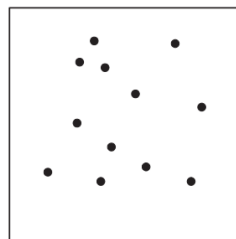
KD-Tree



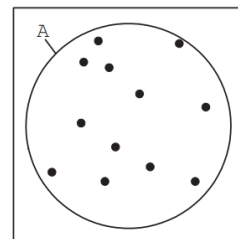
¿K-NN?



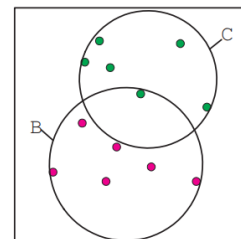
Ball-Tree



1a. A dataset

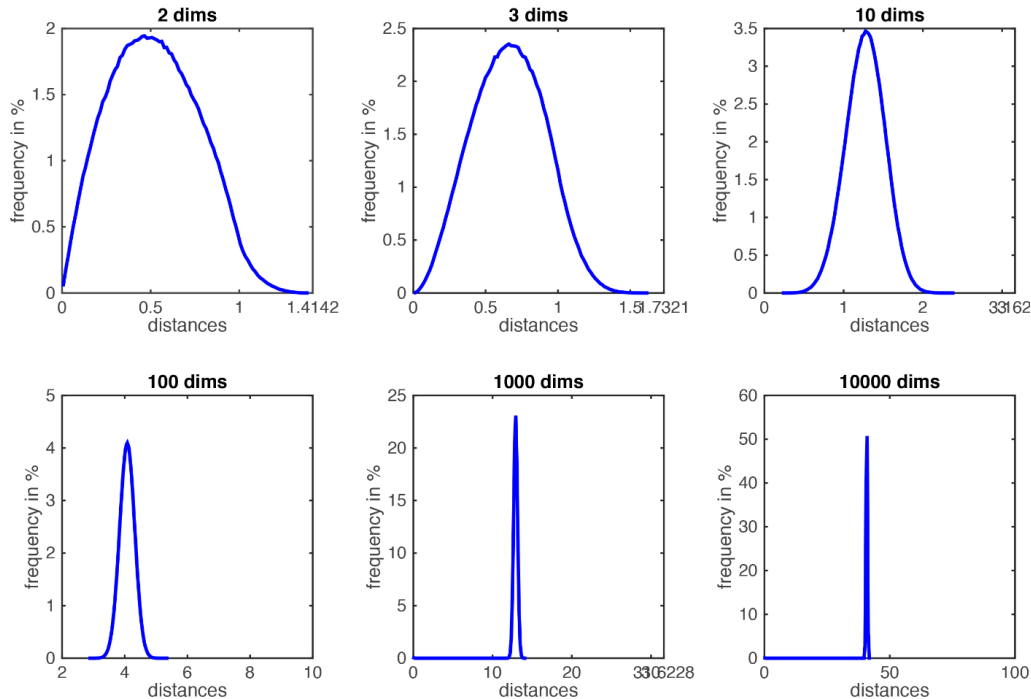


1b. Root node



1c. The 2 children

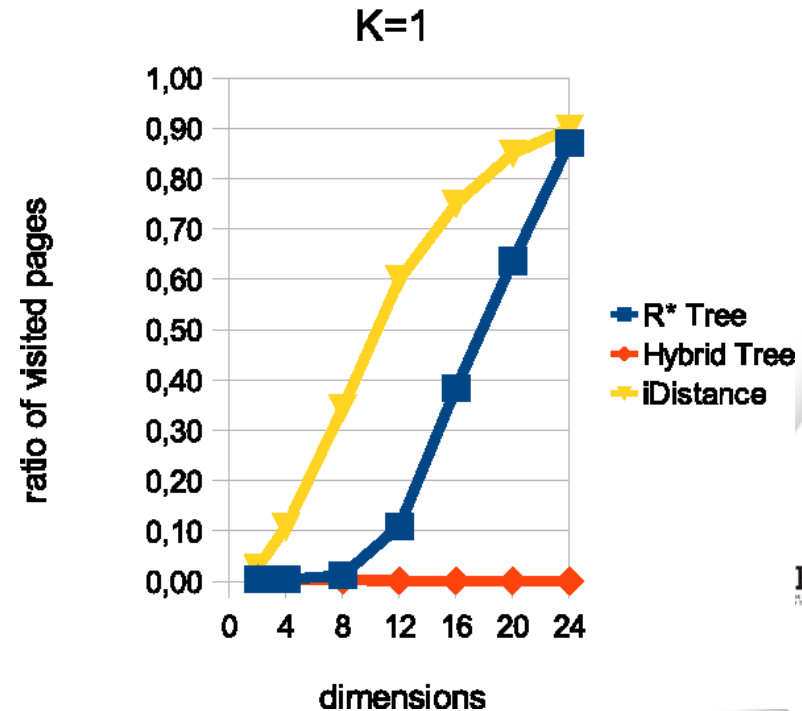
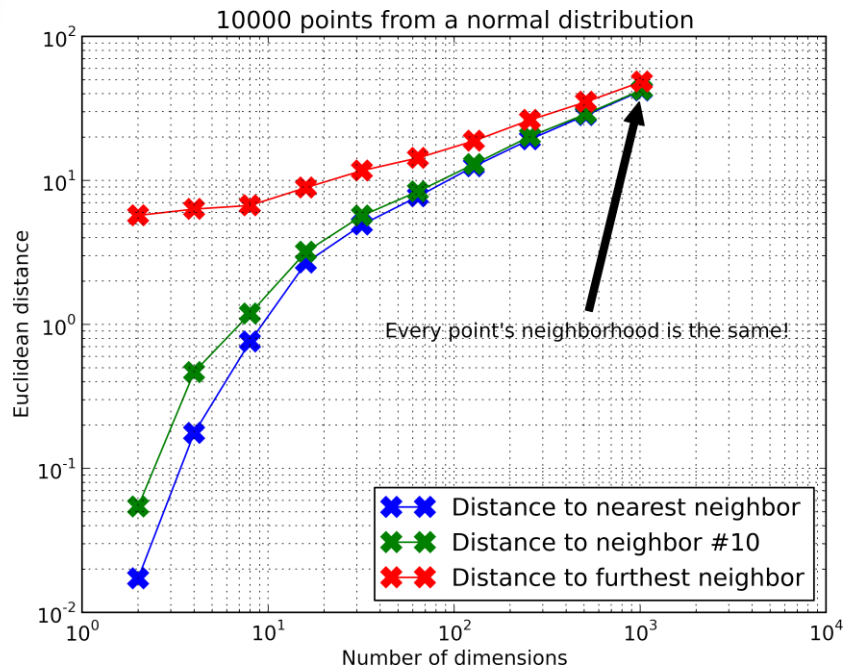
Maldición de Dimensionalidad



Los histogramas muestran la distribución de las distancias calculadas para N pares de puntos distribuidos aleatoriamente en un espacio vectorial unitario.

Se observa que a medida que crece el **número de dimensiones**, la mayor cantidad de **distancias se concentran dentro de un rango muy pequeño**.

Maldición de Dimensionalidad



Kourioukidis, Nikolaos and Georgios Evangelidis. "The Effects of Dimensionality Curse in High Dimensional kNN Search." 2011 15th Panhellenic Conference on Informatics (2011): 41-45.

5.

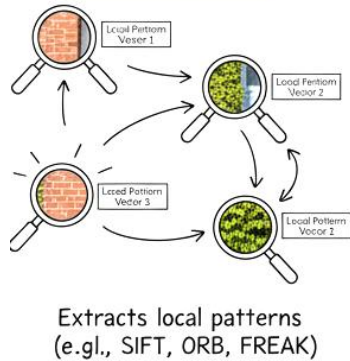


Modelos de Recuperación Multimedia

Descriptores Locales

Descriptores Locales

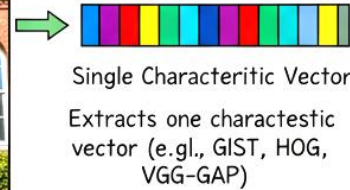
LOCAL DESCRIPTORS



ANY IMAGE



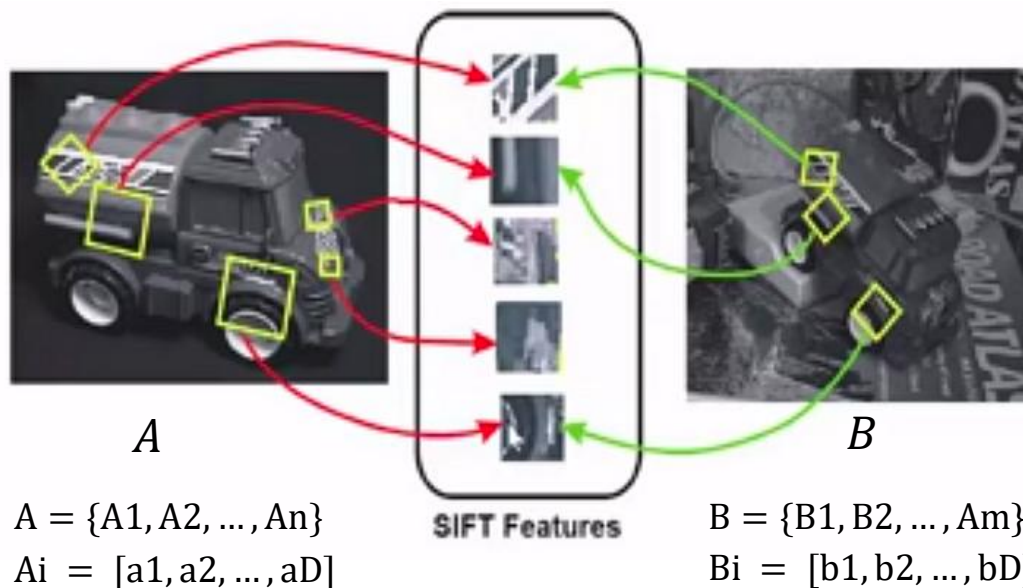
GLOBAL DESCRIPTOR



FEATURE EXTRACTION STRATEGIES

Descriptores Locales: SIFT

Detección de descriptores locales invariantes a la escala y a la rotación.

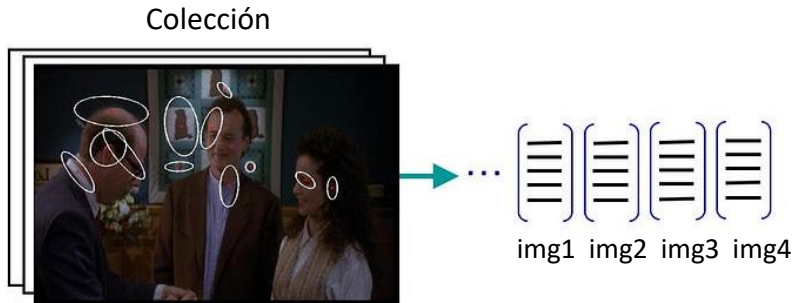


Se generan una cantidad diferente de vectores característicos para cada imagen.

$\delta(SIFT(A), SIFT(B)) ??$

Descriptores Locales: SIFT

- SIFT descriptor



```
import cv2

# Cargar imagen en escala de grises
image = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)
# Crear detector SIFT
sift = cv2.SIFT_create()
# Detectar keypoints y descriptores
keypoints, descriptors = sift.detectAndCompute(image, None)
# Mostrar cantidad de puntos clave y tamaño de descriptores
print(f"Número de keypoints detectados: {len(keypoints)}")
print(f"Forma del array de descriptores: {descriptors.shape}") # (n_puntos, 128)
```

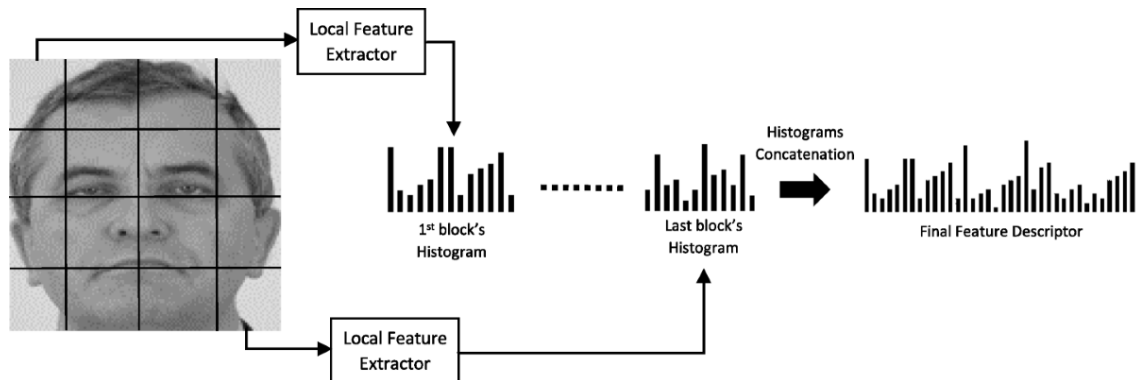
Descriptores Locales: Similitud

$$\delta(SIFT(A), SIFT(B)) ??$$

Para la medir la similitud, se puede usar los descriptores locales de **forma agregada** o **directamente**

- Histograma de Cuantización (k-means)
- Árboles de Cuantización (Hierarchical K-Means)
- Índice Invertido para Descriptores Locales

- Índice Multidimensionales para Descriptores Locales
- Índices de Alta Dimensión para Búsqueda Aproximada
- Búsqueda Lineal con Distancia Apropiaada (DTW, ED, etc)



Descriptores Locales: Condiciones

- Implica segmentar el problema en subespacios más pequeños.
- **Los descriptores locales deben ser baja dimensión**
- Se puede usar un solo índice para todos los descriptores locales, o también un índice diferente para cada descriptor local.
 - **Depende la naturaleza del descriptor.**
- El objeto de consulta también se le aplica extracción de descriptores locales y aplicar KNN a cada descriptor local,
- Luego se debe aplicar alguna técnica efectiva de **combinación de resultados parciales** para obtener el resultado final.
- El resultado no será exactamente igual a una búsqueda lineal, por eso esta clasificado como **método de búsqueda aproximada**.

Descriptores Locales: Ventajas

- **Eficiencia Computacional:** Son más rápidos de calcular y comparar, ya que se enfocan en regiones específicas de la imagen.
- **Robustez a la Oclusión:** Son útiles en escenarios donde partes de la imagen pueden estar ocultas o no estar presentes en todas las imágenes.
- **Flexibilidad:** Pueden adaptarse a diferentes tipos de características y pueden ser combinados para mejorar la precisión en la detección y descripción de puntos de interés.

Descriptores Locales: Desventajas

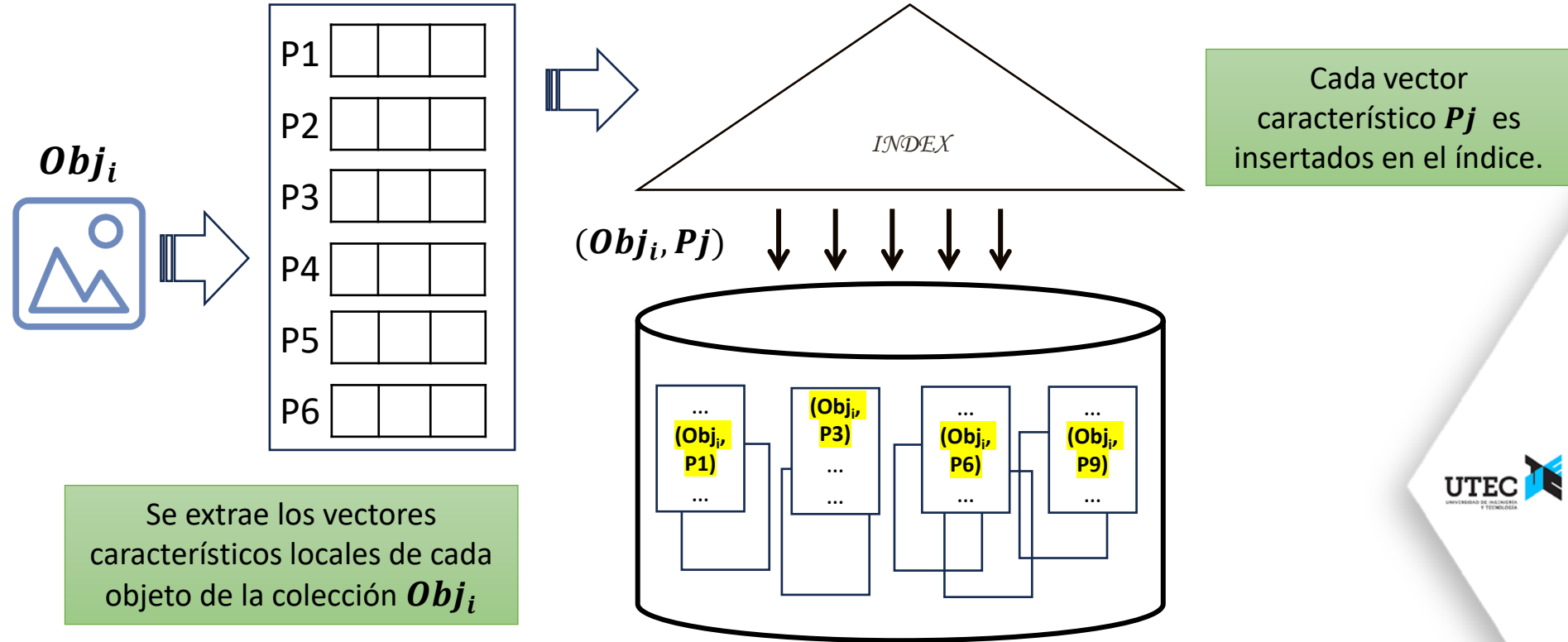
- **Alta Dimensionalidad:** Los descriptores locales como SIFT generan vectores de alta dimensión, 128, lo cual sigue siendo un desafío en recursos computacionales.
- **Sensibilidad al Ruido:** Pueden ser sensibles al ruido en el objeto, esto puede llevar a descripciones inexactas o inconsistentes, afectando a la precisión.
- **Escalabilidad:** La extracción de descriptores locales puede ser costosa en términos de recursos computacionales, por lo que, escalar su uso a grandes volúmenes de datos puede ser un desafío.

Manejar y procesar **grandes bases de datos** de descriptores puede requerir técnicas de optimización, como los **métodos de búsqueda aproximada**.

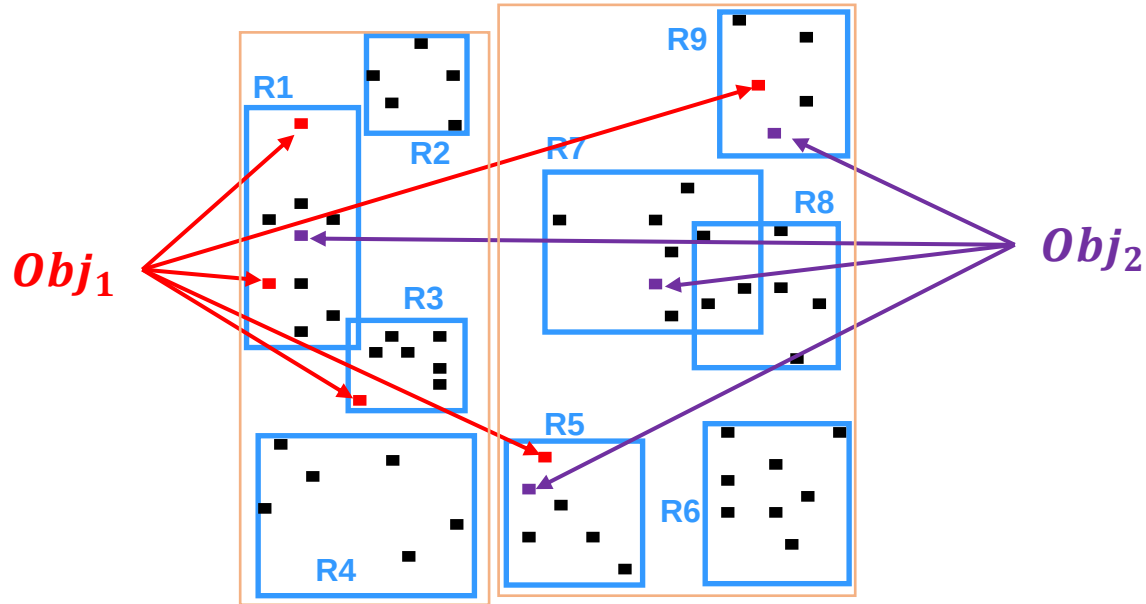
Índice Multidimensional con Descriptores Locales

UTEC

Indexación de Descriptores Locales: Un solo Índice



Indexación de Descriptores Locales: Un solo Índice



Los vectores característicos P_j de cada objeto de la colección quedan ubicados en diferentes MBRs.

Indexación de Descriptores Locales: Búsqueda

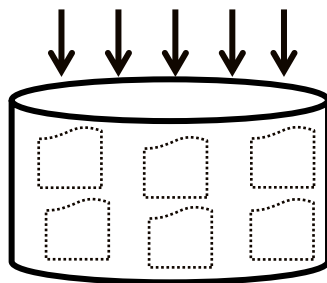
query

Q1			
Q2			
Q3			
Q4			
Q5			
Q6			

Se extrae los vectores
característicos locales

Filter

INDEX



3NN-search

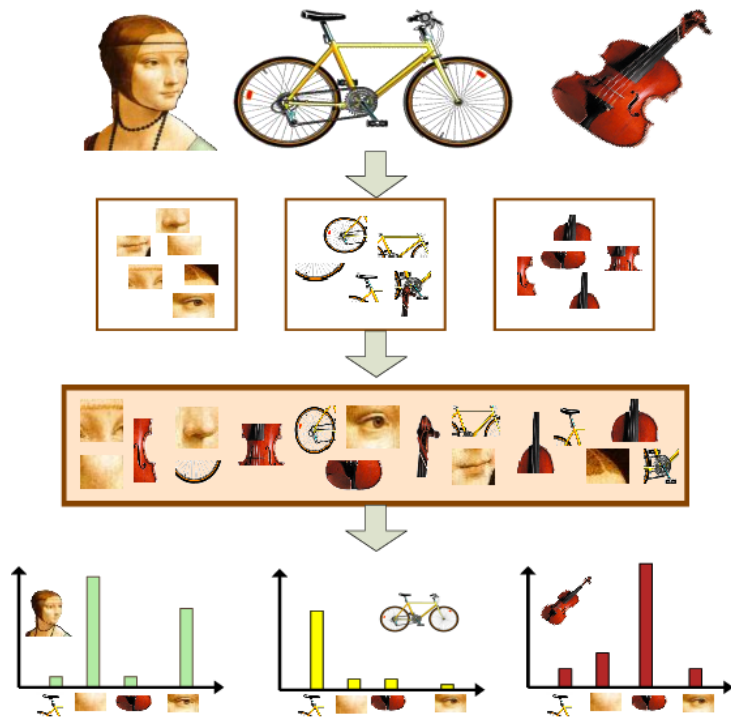
Q1	obj4	obj2	obj6
Q2	obj1	obj4	obj2
Q3	obj4	obj8	obj2
Q4	obj2	obj4	obj6
Q5	obj4	obj3	obj2
Q6	obj1	obj8	obj4

Aplicar una estrategia para combinar
todos los resultados parciales y
obtener los k vecinos cercanos finales.

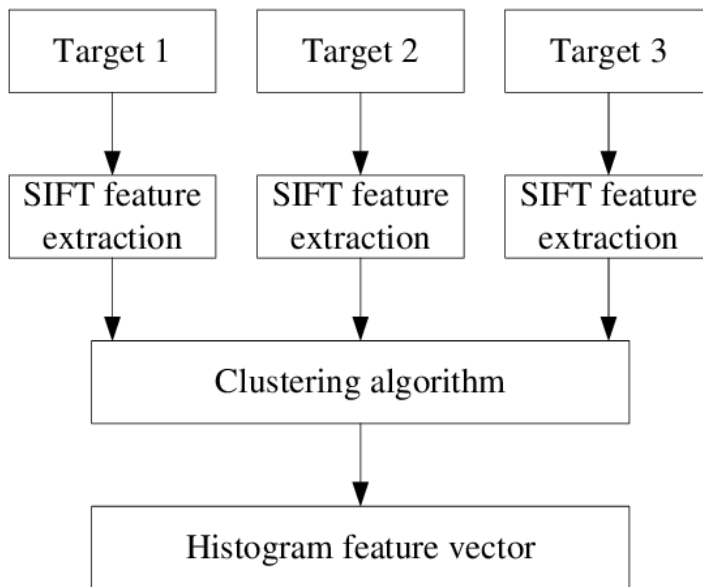
Cuantización de Descriptores Locales



Bag of Visual Words: Construcción



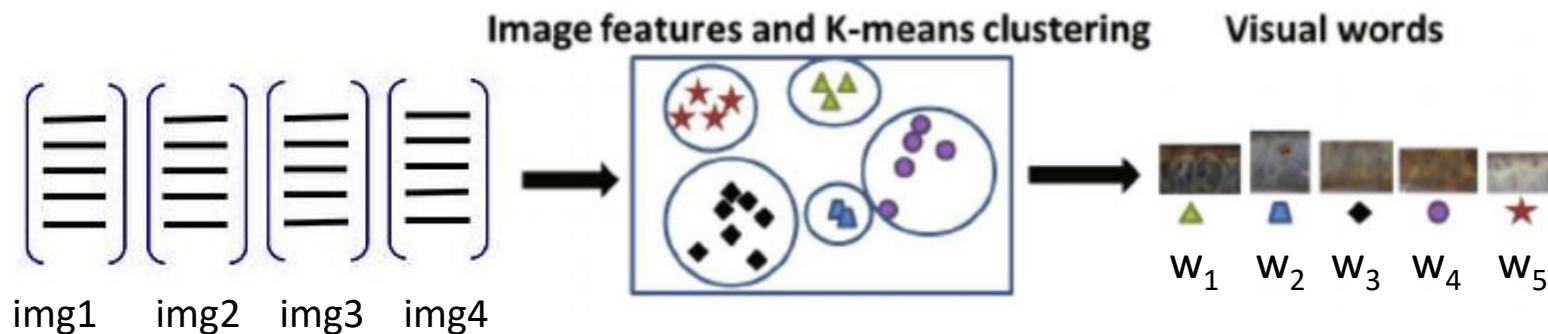
(a) Algorithm diagram



(b) Algorithm flowchart

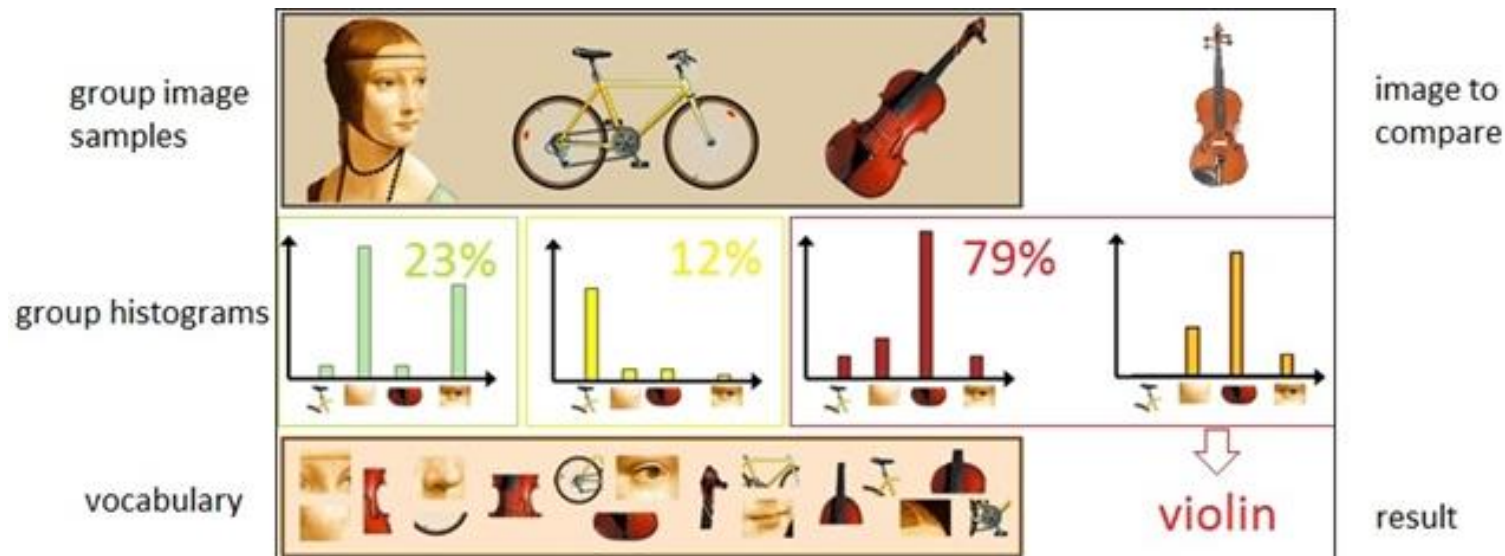
Bag of Visual Words: Clustering

- Clustering Algorithm

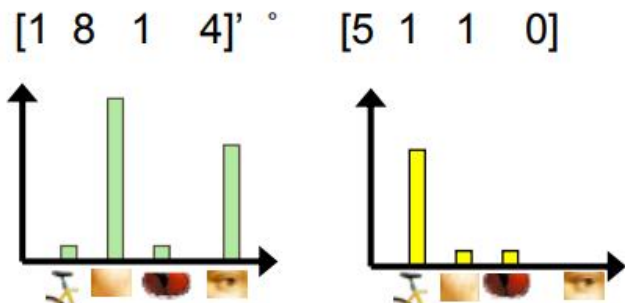


[Visualizing K-Means Clustering](#)

Bag of Visual Words: Consulta



Bag of Visual Words: Consulta



$\vec{p}$



$\vec{q}$

$$\text{Similarity}(p, q) = \cos \theta = \frac{p \cdot q}{\|p\| \|q\|} = \frac{\sum_{i=1}^n p_i q_i}{\sqrt{\sum_{i=1}^n p_i^2} \sqrt{\sum_{i=1}^n q_i^2}}$$

Aplicar TF-IDF

$$p_i = \underbrace{\frac{tf_{i,p}}{|p|}}_{\log(1 + tf_{i,p})} \log \frac{N}{df_i}$$

Índice Invertido con Descriptores Locales



Inverted Index (IVF): Construcción

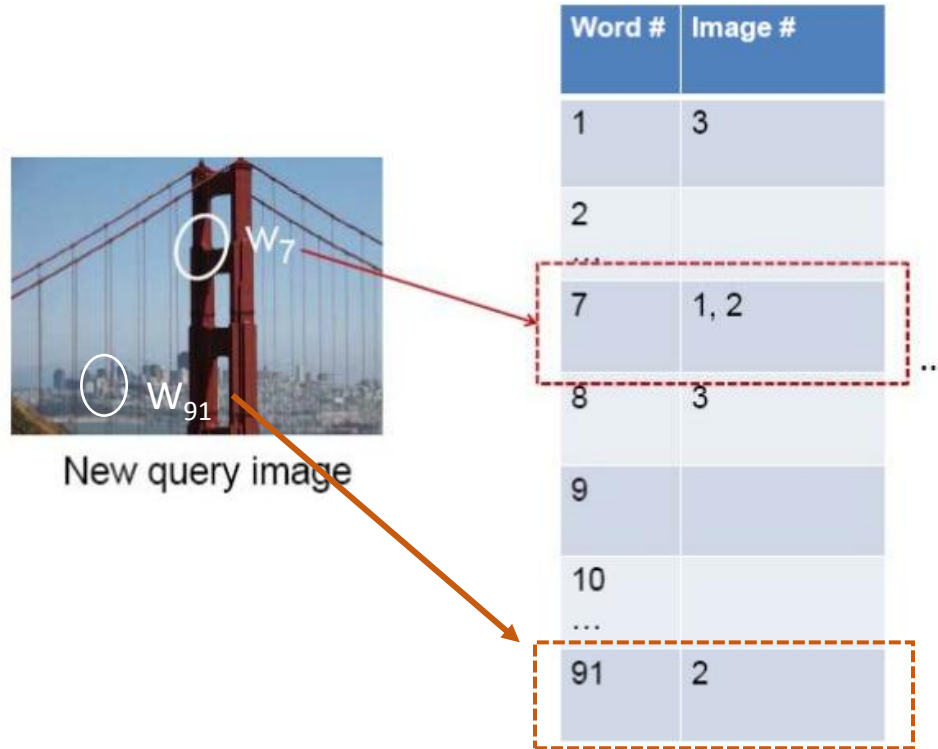
Database images		Image #1	
		Image #2	
		Image #3	

Word #	Image #
1	3
2	
...	
7	1, 2
8	3
9	
10	
...	
91	2

Inverted Index: Construcción

1. **Muestreo:** Se recopila todos los descriptores locales (e.g., SIFT) de un gran conjunto de imágenes de entrenamiento (base de datos).
2. **Clustering (K-Means):** Se aplica un algoritmo de *clustering* (e.g. K-Means) a este conjunto de descriptores. Los centroides obtenidos conforman el Vocabulario Visual.
3. **Asignación de Descriptores:** Para cada imagen de la base de datos, se toma cada descriptor local individualmente y se le asigna a la Palabra Visual (centroide) más cercano. Esta es la cuantización.
4. **Indexación:** El Índice Invertido se construye utilizando los IDs de las Palabras Visuales como claves.
 - **Posting Lists:** Para cada clave (Palabra Visual), se almacena una lista de todas las Imágenes que contienen descriptores cuantizados a ese centroide.

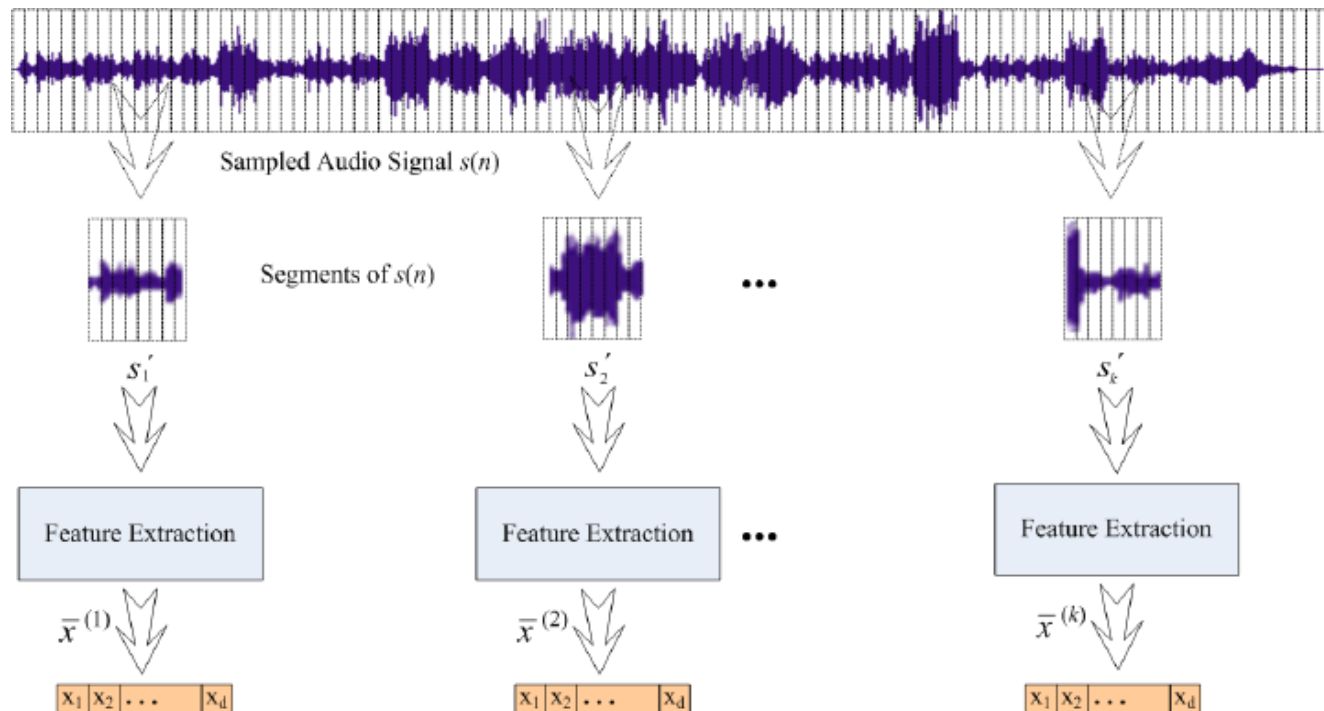
Inverted Index: Búsqueda



Inverted Index: Búsqueda

1. **Consulta:** Se genera los descriptores locales y se cuantizan de la misma manera. Esto genera una lista de Palabras Visuales que contiene la imagen de consulta.
2. **Filtrado:** Se utiliza el Índice Invertido para recuperar solo las imágenes que comparten al menos una de estas Palabras Visuales.
3. **Similitud de Coseno:**
 - Para las imágenes recuperadas, se construye o se recupera su **Histograma BoVW** ponderado (**TF-IDF**).
 - Se aplica la **Similitud de Coseno** entre el Histograma BoVW de la consulta y los Histograma BoVW de las imágenes candidatas para determinar el *ranking* de similitud.

¿Para audio?: segmentación con ventanas deslizantes



Se generan
vectores locales
más pequeños.

Recuperación Eficiente en Texto

1) INDEXACIÓN (Offline)

1. Colección de documentos



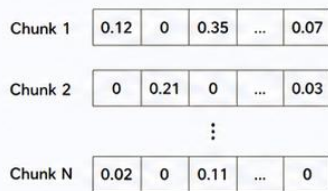
Documentos de texto

2. Procesar texto por chunks



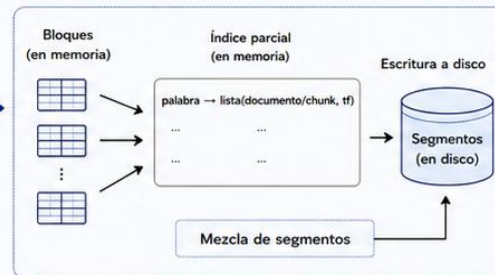
Dividir documentos en fragmentos manejables (chunks)

3. Representación vectorial (TF-IDF / Bag of Words)



Cada chunk se convierte en un vector TF-IDF (Bag of Words)

4. Indexación (SPIMI - Memoria secundaria)



SPIMI: se construyen índices parciales en memoria y se escriben segmentos en disco; luego se mezclan.

5. Índice invertido final (en disco)

Palabra	Lista de (documento/chunk, tf)
texto	(doc3/ch3, 2), (doc7/ch1, 1), ...
información	(doc1/ch2, 3), (doc3/ch1, 1), ...
filtro	(doc2/ch1, 2), (doc5/ch3, 1), ...
...	...

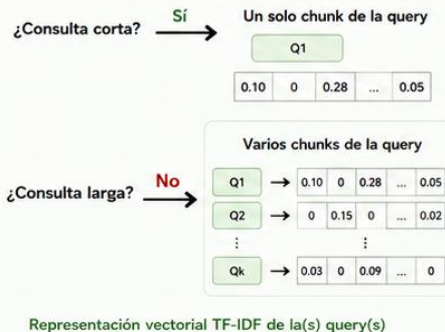
Cada palabra se asocia a la lista de documento/chunk donde aparece y su frecuencia (tf). Se almacena idf global por término.

2) RECUPERACIÓN (Online)

1. Consulta del usuario (query)

¿Qué es un filtro basado en el contenido?

2. Procesar la query



3. Búsqueda en índice invertido (Recuperación de candidatos)

- Para cada término de la query:
- Buscar en el índice invertido
 - Obtener lista de documento/chunk y tf
 - Calcular ponderación TF-IDF del documento/chunk

Palabra	Lista de (documento/chunk, tf)
filtro	(doc2/ch1, 2), (doc5/ch3, 1), ...
basado	(doc1/ch2, 1), (doc4/ch7, 2), ...
contenido	(doc3/ch1, 2), (doc1/ch3, 1), ...
...	...

4. Cálculo de similitud de coseno

- q = vector query
- d = vector documento/chunk candidato

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

donde:

- $q \cdot d = \sum_{i=1}^{|V|} q_i d_i$ (producto punto)
- $\|q\| = \sqrt{\sum_{i=1}^{|V|} q_i^2}$ (norma)
- $\|d\| = \sqrt{\sum_{i=1}^{|V|} d_i^2}$ (norma)

5. Ranking y combinación

Ordenar candidatos por $\text{sim}(q, d)$ descendente

Rank	Documento/Chunk	$\text{sim}(q, d)$
1	(doc3 / ch1)	0.892
2	(doc1 / ch2)	0.763
3	(doc5 / ch3)	0.641
4	(doc2 / ch1)	0.487
5	(doc7 / ch4)	0.352
...	...	...

Si la query es larga (varios chunks $Q_1..Q_k$):

- Calcular $\text{sim}(Q_i, d)$ para cada Q_i
- Combinar scores (p.ej., PROMEDIO o MÁX):
$$\text{score}(d) = \frac{1}{k} \sum_{i=1}^k \text{sim}(Q_i, d)$$
- Re-ranquear por $\text{score}(d)$

Notas:

- TF: frecuencia del término en el chunk/documento
- IDF: $\log\left(\frac{N}{df_t}\right)$ (N: # de documentos, df_t : # de documentos/chunks que contienen el término t)

• SPIMI: Single-Pass In-Memory Indexing (índice en memoria construido con escritura incremental en disco)

• Ventaja del coseno: normaliza por la longitud de los vectores, permitiendo comparar consultas y documentos/chunks de distinto tamaño.

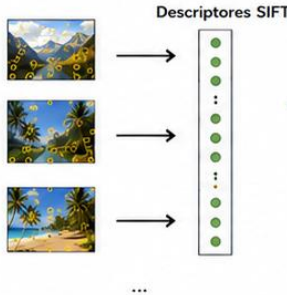
Recuperación Eficiente en Imágenes

1) INDEXACIÓN (Offline)

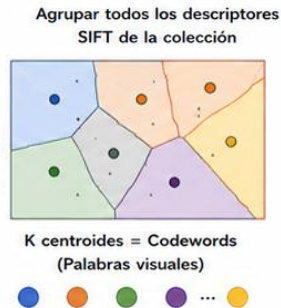
1. Colección de imágenes



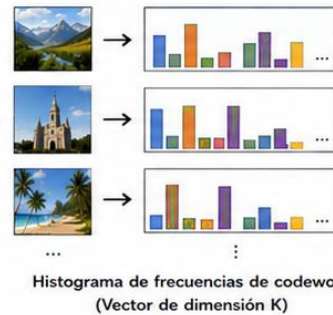
2. Extracción de descriptores locales (SIFT)



3. Aprendizaje de codewords (K-Means)



4. Representación de cada imagen (Histograma de cuantización)



5. Indexación (Índice invertido)

Para cada codeword (centroide) se almacenan las imágenes que lo contienen y su frecuencia (tf) en el histograma.

Codeword (Centroide)	Lista de imágenes (img_id, tf)
●	(img_2, 3), (img_5, 1), (img_9, 4), ...
●	(img_1, 2), (img_2, 5), (img_7, 1), ...
●	(img_1, 4), (img_3, 2), (img_8, 3), ...
●	(img_2, 1), (img_4, 2), (img_9, 3), ...
●	(img_3, 1), (img_6, 2), (img_10, 1), ...
...	...

Índice invertido sobre codewords

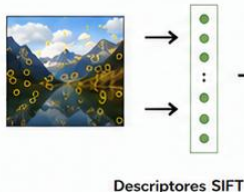
Resultado de indexación: cada imagen representada por un histograma de codewords y almacenada en un índice invertido.

2) BÚSQUEDA (Online)

1. Imagen de consulta

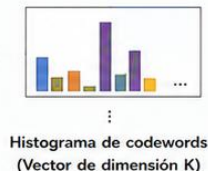


2. Extracción de descriptores (SIFT)



3. Cuantización y representación

Asignar cada descriptor al codeword (centroide) más cercano



4. Búsqueda en el índice invertido (Recuperación de candidatos)

Para cada codeword con frecuencia > 0 en la consulta:

- Buscar en el índice invertido
- Recuperar todas las imágenes que contienen ese codeword (tf > 0)

Conjunto de candidatos

Unión de todas las imágenes recuperadas (que comparten al menos un codeword con la consulta)

5. Cálculo de similitud (Similitud de coseno)

q = vector consulta (histograma)
d = vector imagen candidata (histograma)

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

donde:

$$q \cdot d = \sum_{i=1}^K q_i d_i$$
$$\|q\| = \sqrt{\sum_{i=1}^K q_i^2}, \quad \|d\| = \sqrt{\sum_{i=1}^K d_i^2}$$

6. Ranking y resultados

Ordenar por similitud descendente

Rank	Imagen	sim(q, d)
1	img_2	0.892
2	img_5	0.763
3	img_1	0.641
4	img_3	0.487
5	img_7	0.352
...	...	...



SIFT: Scale Invariant Feature Transform (descriptores locales invariantes a escala y rotación)
K-Means: algoritmo de clustering para obtener K centroides (codewords)

Codeword: palabra visual (centroide)
Histograma (vector de codewords): distribución de frecuencias de los codewords en la imagen (dimensión K)

Índice invertido: asocia cada codeword con las imágenes que lo contienen y su frecuencia (tf) en el histograma

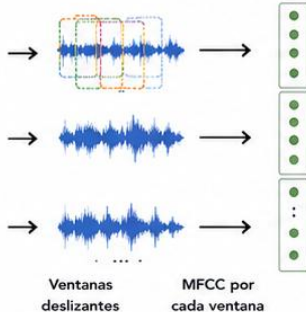
Recuperación Eficiente en Audio

1) INDEXACIÓN (Offline)

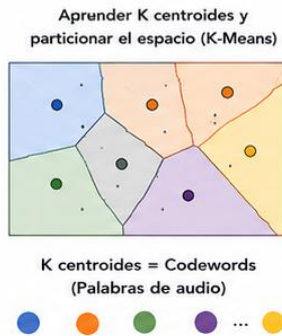
1. Colección de audios (Colección)



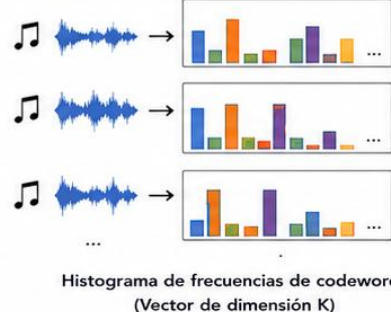
2. Extracción de características (Sliding Window + MFCC)



3. Aprendizaje de codewords (K-Means)



4. Representación de cada audio (Histograma de frecuencias)



5. Indexación (Índice invertido)

Para cada codeword (centroide) se almacenan los audios que lo contienen y su frecuencia (tf) en el histograma.

Codeword (Centroide)	Lista de audios (audio_id, tf)
●	(audio_12, 3), (audio_45, 1), (audio_7, 4), ...
●	(audio_3, 2), (audio_18, 5), (audio_9, 1), ...
●	(audio_7, 1), (audio_23, 3), (audio_45, 2), ...
●	(audio_12, 1), (audio_3, 4), (audio_30, 3), ...
●	(audio_9, 3), (audio_18, 2), (audio_5, 10), ...
...	

Índice invertido sobre codewords

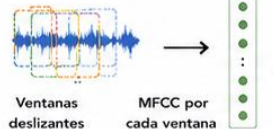
Resultado de indexación: cada audio representado por un histograma de codewords y almacenado en un índice invertido.

2) BÚSQUEDA (Online)

1. Audio de consulta (Query)

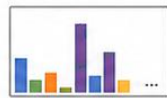


2. Extracción de características (Sliding Window + MFCC)



3. Cuantización (Asignación a codewords)

Asignar cada vector MFCC al codeword (centroide) más cercano



Histograma de codewords de la query (Vector de dimensión K)

4. Búsqueda en el índice invertido (Recuperación de candidatos)

Para cada codeword con frecuencia > 0 en la query:

- Buscar en el índice invertido
- Recuperar audios que contienen ese codeword (tf > 0)

Conjunto de candidatos

Unión de todos los audios recuperados (que compartan al menos un codeword con la query)

5. Cálculo de similitud (Similitud de coseno)

q = vector query (histograma)

d = vector audio candidato (histograma)

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

donde:

$$q \cdot d = \sum_{i=1}^K q_i d_i$$

$$\|q\| = \sqrt{\sum_{i=1}^K q_i^2}, \quad \|d\| = \sqrt{\sum_{i=1}^K d_i^2}$$

6. Ranking y resultados (Orden descendente)

Rank	Audio	Similitud
1	audio_12	0.912
2	audio_45	0.784
3	audio_7	0.653
4	audio_18	0.512
5	audio_3	0.401
...		...



Sliding Window: ventana temporal
MFCC: Mel Frequency Cepstral Coefficients

Codeword: palabra de audio (centroide)
K-Means: algoritmo de clustering para obtener K centroides

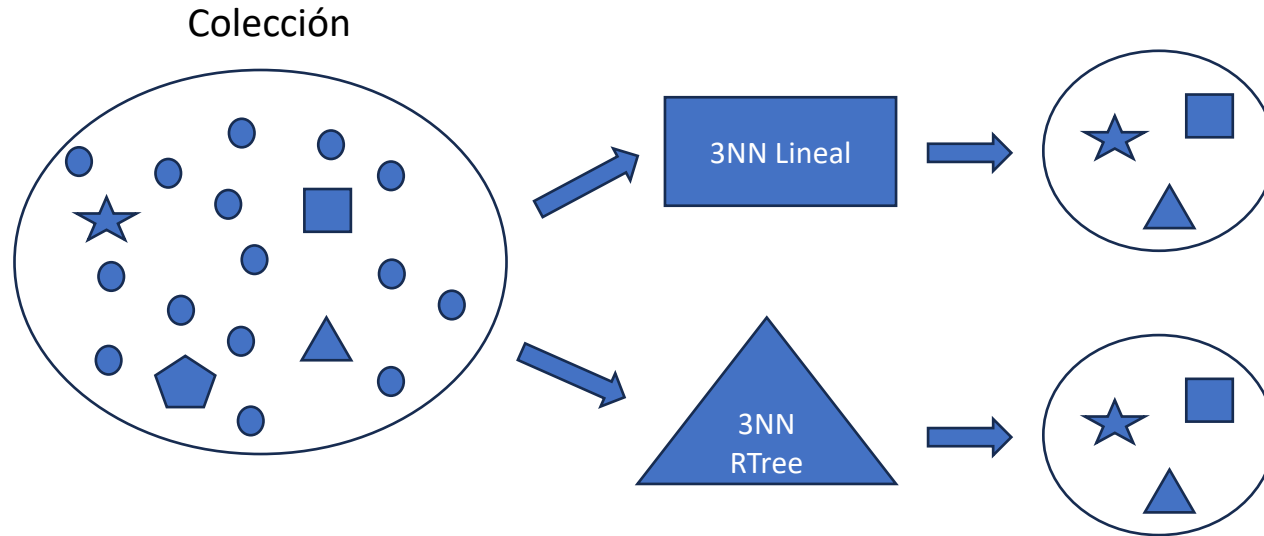
Histograma de codewords: distribución de frecuencias de los codewords en un audio (dimensión K)

Índice invertido: asocia cada codeword con los audios que lo contienen y su frecuencia (tf) en el histograma



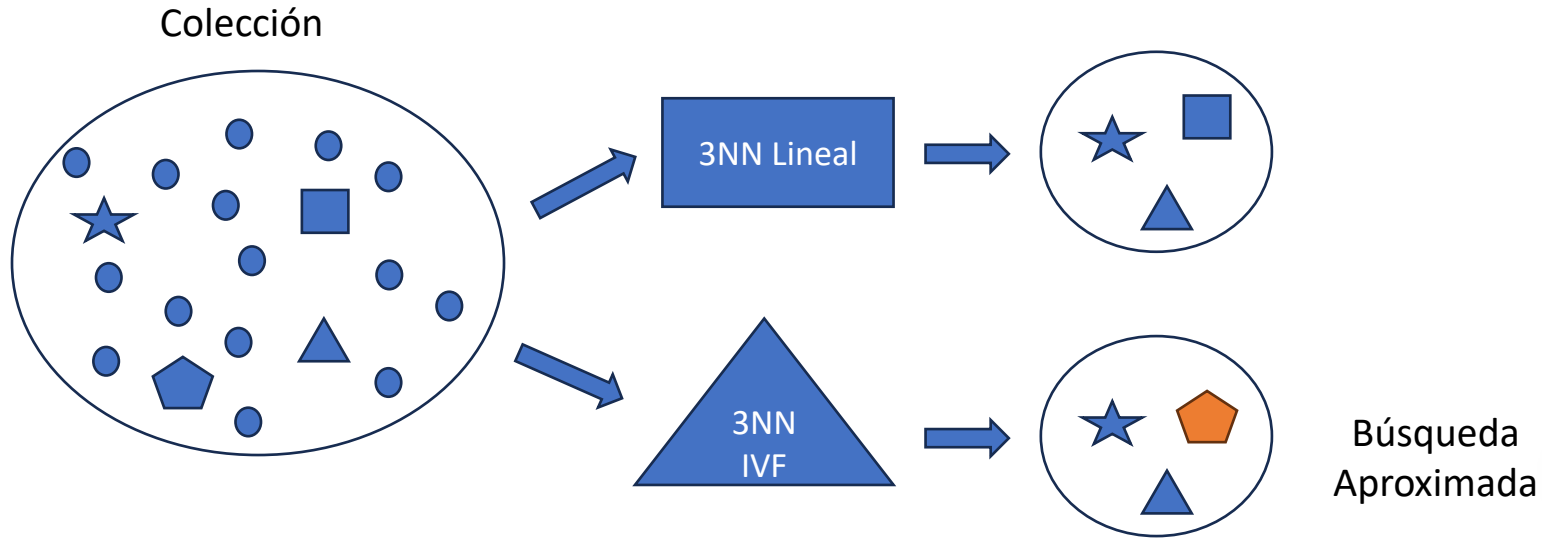
Búsqueda Aproximada ANN - Approximate Nearest Neighbors

ANN - Approximate Nearest Neighbors



Búsqueda
Exacta

ANN - Approximate Nearest Neighbors



Los métodos de búsqueda aproximada son muy eficientes al trabajar en altas dimensiones. Sacrifican precisión por eficiencia.

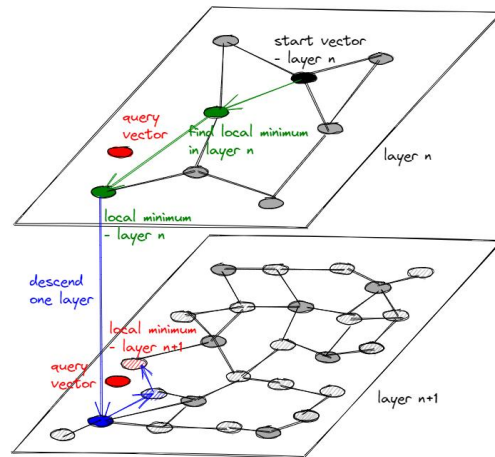
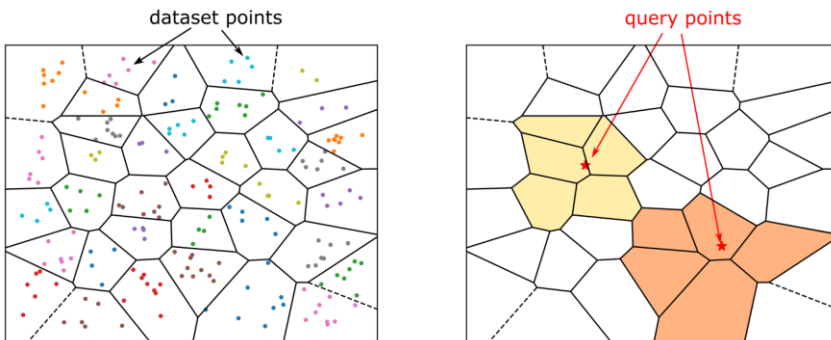
Búsqueda Lineal vs Búsqueda Aproximada

Característica	Búsqueda Lineal	Búsqueda Aproximada
Definición	Compara directamente cada elemento con la consulta	Utiliza técnicas para encontrar resultados "suficientemente buenos"
Complejidad	$O(n)$	Menor complejidad comparada con la búsqueda lineal
Exactitud	Encuentra el resultado exacto	Sacrifica precisión por eficiencia
Aplicación	Eficiente en conjuntos de datos pequeños	Ideal para conjuntos de datos grandes y de alta dimensionalidad
Ventaja	Precisión absoluta	Maneja mejor la maldición de la dimensionalidad
Desventaja	Ineficiente en datos grandes y alta dimensionalidad	Pequeña pérdida de precisión

Técnicas de Búsqueda Aproximada:

- **IVF Flat:** Índice invertido con compresión plana, trabaja con **descriptores globales**, rápido de construir, eficiente en memoria, menos preciso que HNSW.
- **HNSW:** Algoritmo basado en **grafos jerárquicos**, más lento de construir, pero más preciso y rápido en la búsqueda, puede requerir más memoria.

Investigar su funcionamiento



Implementaciones

- PostgreSQL

PG Vector (IVF, HNSW)

Support kNN search

- Python

scikit-learn (LSHForest)

Support range and kNN search

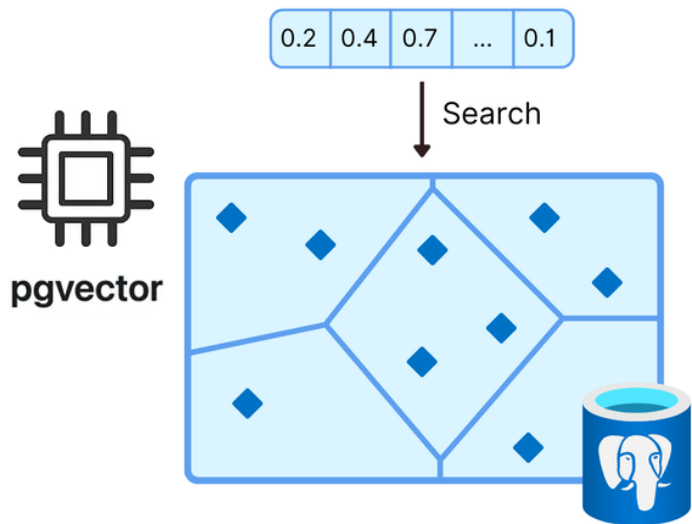
Faiss (IVF, HNSW, LSH)

Support kNN search

Support GPU

PostgreSQL como Vector Database

PostgreSQL puede almacenar y gestionar **vectores característicos** eficientemente usando la extensión **pgvector**. Soporta métricas como **distancia euclidiana** y **similitud del coseno**. Además, índices de alta dimensionalidad como **IVFFlat** y **HNSW**.



CREATE EXTENSION vector;

Operador	Métrica	Descripción	Rango
$\langle - \rangle$	Euclidiana (L2)	Distancia línea recta	$[0, \infty]$
$\langle \# \rangle$	Negación Producto Punto	Opuesto del dot product	$(-\infty, \infty]$
$\langle = \rangle$	Coseno (Angular)	Ángulo entre vectores	$[0, 1]$
$\langle @ \rangle$	Inclusión	Componentes dentro	bool
$@ \rangle$	Contenedor	Contiene componentes	bool
$ $	Norma L2	Magnitud del vector	$[0, \infty]$

[Use IVFFlat index on Azure Cosmos DB for PostgreSQL for similarity search](#)

pgvector: Sintaxis SQL Práctica

1. Crear tabla:

```
CREATE TABLE documents (  
  id SERIAL PRIMARY KEY,  
  content TEXT,  
  embedding vector(768)  
);
```

2. Búsqueda por similitud:

```
SELECT id, content FROM documents  
ORDER BY embedding <-> '[0.1, 0.2, ...]':vector  
LIMIT 10;
```

3. Crear índice IVFFLAT:

```
CREATE INDEX ON documents  
USING ivfflat (embedding vector_cosine_ops)  
WITH (lists = 100);
```

4. Crear índice HNSW

```
CREATE INDEX ON documents  
USING hnsu (embedding vector_cosine_ops)  
WITH (m = 16, ef_construction = 64);
```

Laboratorio 10

